

Calcolatori Elettronici

16 febbraio 2023

Teoria

Esercizio 1 (12 punti): Si sintetizzi una rete sequenziale sincrona che riconosca se il numero di 1 in una stringa di 4 bit è pari (P) o dispari (D). La rete interpreta la sequenza di ingresso come una successione di stringhe di lunghezza 4 e dà la risposta in corrispondenza del quarto bit di ogni stringa. La sequenza di input (x) è infinita. Nel mezzo di una parola di 4 bit, l'uscita corrisponde al valore ? indicante "sottosequenza non completa". Ad esempio:

1	x:	1 0 0 1		0 0 1 0		0 0 0 0		1 1 1 1		1 0 1 1		...
2	z:	? ? ? P		? ? ? D		? ? ? P		? ? ? P		? ? ? D		...

Realizzare il circuito che calcola il valore dell'uscita z.

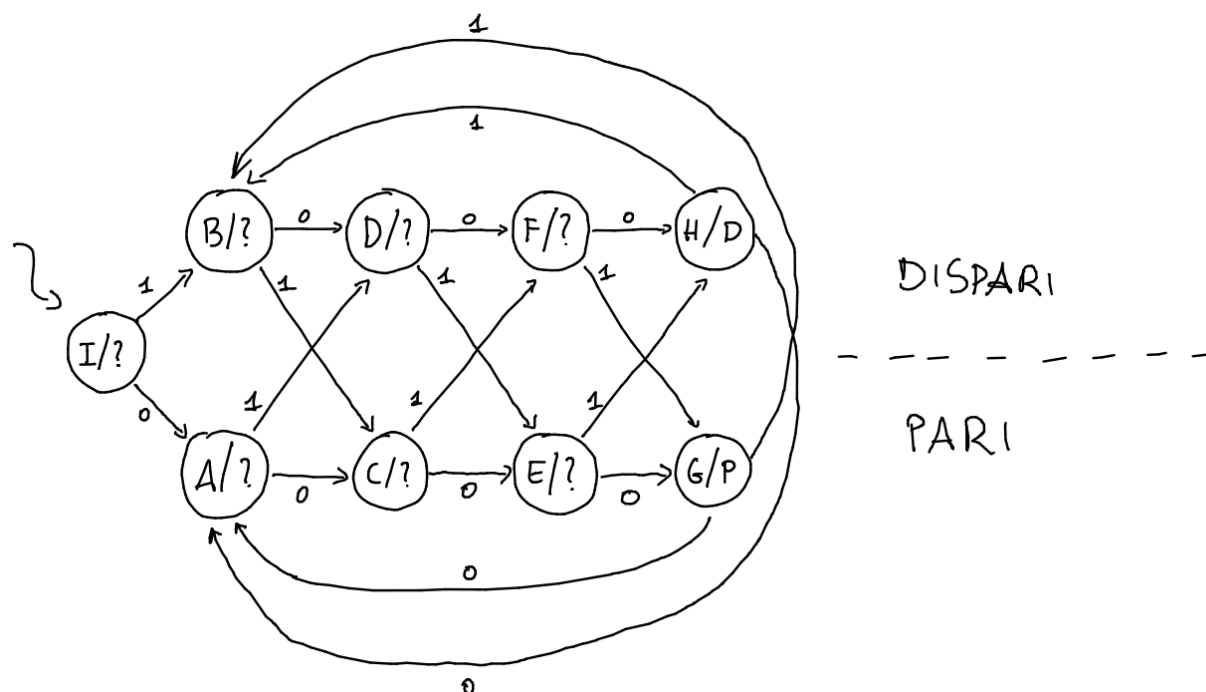
Esercizio 2 (12 punti): Disegnare l'architettura interna del processore z64, dopodiché descrivere la parte della macchina sequenziale (microcodice) atta all'esecuzione dell'istruzione: `addb %a1, %c1`, considerando anche la fase di fetch.

Esercizio 3 (6 punti): Illustrare vantaggi e svantaggi prestazionali dell'architettura a pipeline, confrontandola con l'architettura multiciclo.

Soluzioni

Esercizio 1

La macchina a stati si può vedere come divisa in due gruppi di stati: gli stati A, C, E, G gestiscono un numero pari di 1, mentre gli stati B, D, F, H gestiscono un numero dispari di 1. Lo stato I è lo stato iniziale, il cui scopo è solo quello di consentire di generare l'uscita negli stati G ed H senza dover considerare casi particolari. Ad ogni ricezione di un 1 si transita nell'insieme di stati corretto. La macchina quindi "conta" il numero caratteri letti andando "verso destra" e conta se il numero di 1 è pari o dispari spostandosi tra il gruppo "superiore" e quello "inferiore" di stati.

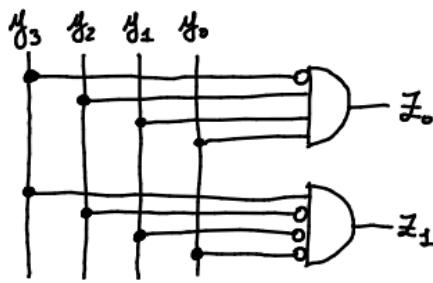


La codifica degli input non è necessaria. Codificando gli stati e gli output si ottiene:

S	y_3	y_2	y_1	y_0
I	0	0	0	0
A	0	0	0	1
B	0	0	1	0
C	0	0	1	1
D	0	1	0	0
E	0	1	0	1
F	0	1	1	0
G	0	1	1	1
H	1	0	0	0

O	z_1	z_0
?	0	0
P	0	1
D	1	0

Avendo utilizzato una macchina di Moore, l'uscita dipende soltanto dallo stato. Poiché l'uscita è P solo nello stato G, si ha: $z_0 = \bar{y}_3 y_2 y_1 y_0$. In maniera del tutto analoga, poiché l'uscita è D solo in H si ha che $z_1 = y_1 \bar{y}_2 \bar{y}_1 \bar{y}_0$. Si noti che la semplicità di queste forme nasce direttamente dall'aver assegnato la codifica 00 al simbolo di uscita ?. Il circuito che calcola l'uscita, quindi, è dato da:



Progetto

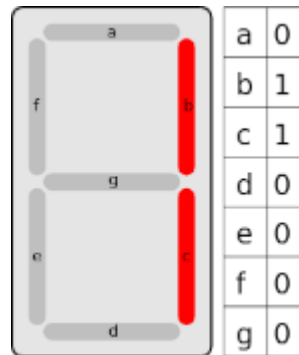
- Un processore z64 gestisce il display che indica la **MARCIA** attualmente ingranata in una motocicletta.

Il processore riceve dalla periferica **CAMBIO** una richiesta di interruzione ogni volta che il guidatore cambia la marcia. Essendo il cambio sequenziale, il processore deve recuperare dal dispositivo **CAMBIO** l'informazione se il guidatore sta scalando la marcia verso l'alto o verso il basso, leggendo il valore di un opportuno registro di interfaccia.

Il dispositivo **MARCIA** è equipaggiato con un display a sette segmenti. I valori che possono essere mostrati sono i seguenti:



dove il primo simbolo (n) indica che la moto è a folle. Il dispositivo **MARCIA** utilizza parole di 7 bit per determinare quali segmenti del led devono essere accesi. Ciascun bit rappresenta lo stato di uno dei segmenti (0 = spento, 1 = acceso), secondo lo schema seguente, dove **a** è il bit meno significativo:



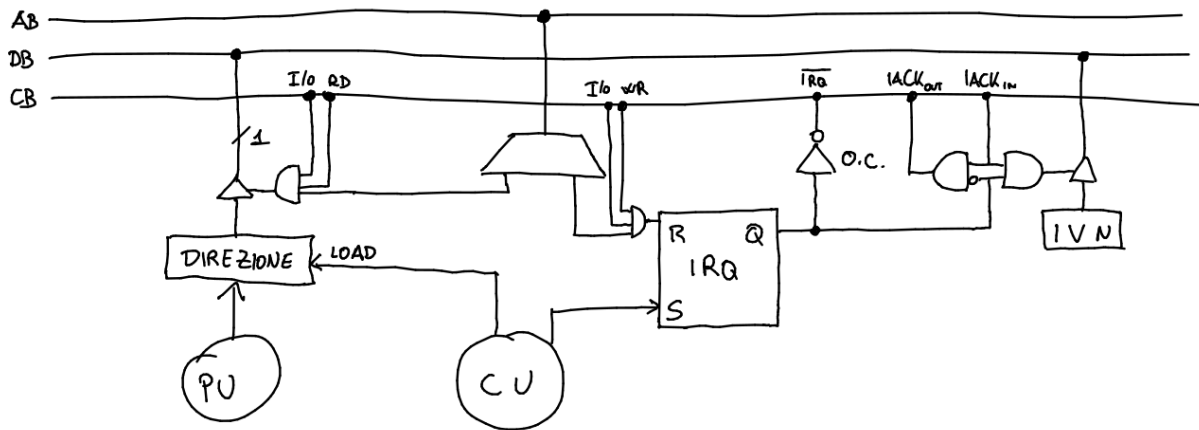
Realizzare:

- Le interfacce dei dispositivi **CAMBIO** e **MARCIA**
- Tutto codice necessario al funzionamento del sistema.

Soluzione

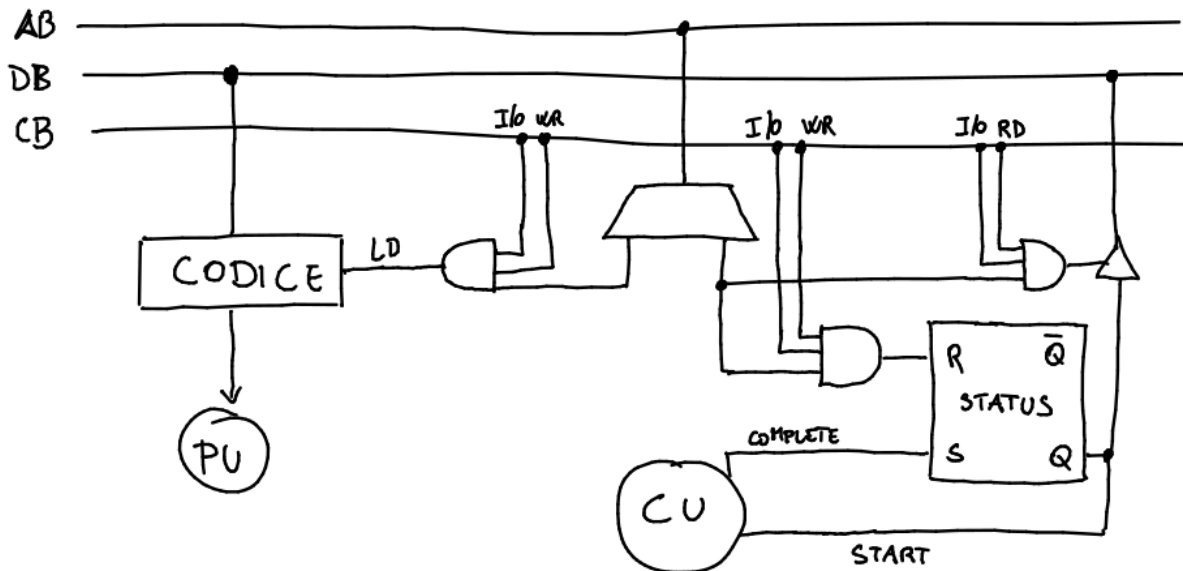
Interfacce delle periferiche:

CAMBIO



DIREZIONE è un registro che assume il valore +1 o -1 per indicare se la marcia è stata scalata verso l'alto o verso il basso. La periferica può generare interruzioni continuamente, poiché non è ragionevole pensare che il software possa impedire al motociclista di muovere il piede. Possiamo comunque assumere senza preoccuparci di compiere errori che la velocità del processore sia enormemente maggiore di quella dell'essere umano.

MARCIA



CODICE è il registro di interfaccia per mantenere la parola di 7 bit (che verrà trattata come byte). Il dispositivo deve essere avviato esplicitamente, per notificare l'avvenuto aggiornamento del contenuto di CODICE.

Una possibile implementazione del software può essere la seguente. Non si effettuano controlli sul valore di marcia perché il cambio sequenziale non permette di raggiungere una marcia non esistente.

```

1  .org 0x800
2  .data
3      marcia: .byte 0 # assumiamo di partire a folle
4      # le codifiche dei valori da mostrare sul display
5      codici: .byte 84, 6, 91, 79, 102, 109
6      .equ M_CODICE, 1

```

```

7      .equ M_STATUS, 2
8      .equ C_DIR, 3
9      .equ C_IRQ, 4
10     .text
11     main:
12         sti
13     .loop:
14         hlt
15         jmp .loop
16
17     .driver 1 # Cambio
18         pushq %rax
19         pushq %rcx
20         inb $C_DIR, %al
21         addb %al, marcia
22         outb %al, $C_IRQ
23         movzbq marcia, %rcx
24         movb codici(, %rcx, 1), %al
25         outb %al, $M_CODICE
26         outb %al, $M_STATUS
27     .bw:
28         inb $M_STATUS, %al
29         btb $0, %al
30         jnc .bw
31         popq %rcx
32         popq %rax
33         iret

```

Prova di laboratorio

Scrivere un programma in C che accetti da tastiera un numero intero di tipo `signed long`. Il programma verifica se il numero inserito è positivo o negativo.

Inoltre, il programma effettua dei controlli sulle rappresentazioni numeriche utilizzate dalla macchina in uso. In particolare, esso verifica se, l'architettura su cui è in esecuzione, utilizza o meno la rappresentazione in complemento a due per rappresentare i numeri negativi.

Inoltre, il programma verifica anche se la rappresentazione in memoria dei dati è in formato big endian o little endian.

Al termine dell'esecuzione, il programma stampa le informazioni ottenute e le rappresentazioni del numero in input nei formati:

- intero decimale;
- esadecimale;
- stringa binaria.

Ad esempio, la seguente è una possibile esecuzione del programma:

Soluzione

```

1  #include <errno.h>
2  #include <limits.h>
3  #include <stdbool.h>
4  #include <stdio.h>
5  #include <stdlib.h>
6
7  bool get_int(long *value)
8  {
9      char buf[1024];
10     bool ret;
11     char *endptr;
12
13     printf("Inserire un numero intero: ");
14     if(!fgets(buf, 1024, stdin)) {
15         return false;
16     }
17
18     errno = 0;
19     *value = strtol(buf, &endptr, 10);
20     if(errno == ERANGE) {
21         printf("Il numero inserito non è rappresentabile.\n");
22         ret = false;

```

```

23     } else if(endptr == buf || (*endptr && *endptr != '\n')) {
24         printf("Input non valido");
25         ret = false;
26     } else {
27         ret = true;
28     }
29
30     return ret;
31 }
32
33 bool twos_complement(void)
34 {
35     return (int)UINT_MAX == -1;
36 }
37
38 bool is_little_endian(void)
39 {
40     unsigned short x = 1;
41     return *(char *) (&x) == 1;
42 }
43
44 #define MAX_BITS (int) (sizeof(long) * CHAR_BIT)
45 static char string[MAX_BITS + 1];
46
47 char *to_binary(long value)
48 {
49     int i;
50     char *p;
51
52     p = string + MAX_BITS;
53     p[0] = 0;
54
55     for(i = 0; i < MAX_BITS; i++) {
56         *--p = value & 1 ? '1' : '0';
57         value >>= 1;
58     }
59     return string;
60 }
61
62 int main(void)
63 {
64     long value;
65     if(!get_int(&value)) {
66         exit(EXIT_FAILURE);
67     }
68
69     printf("Il numero inserito è %s\n", value < 0 ? "NEGATIVO" : "POSITIVO");
70     printf("La rappresentazione%s utilizza il complemento a due.\n", twos_complement() ? "" : " non");
71     printf("La macchina usa la rappresentazione %s ENDIAN\n\n", is_little_endian() ? "LITTLE" : "BIG");
72     puts("Codifiche del valore inserito:");
73     printf("Decimale: %ld\n", value);
74     printf("Esadecimale: %#lx\n", value);
75     printf("Binario: %s\n", to_binary(value));
76
77     return 0;
78 }

```